CS 5800 · ALGORITHMS · SPRING 2026

Real-Time Fashion Trend Detection

Vasudha Jain · Parvathi Gottumukkala · Bhoomika Panday

Advisor: Dr. Lama Hamadi | Khoury College of Computer Sciences | Northeastern University

Why This Problem Matters

10B

Naive Operations

at 100K events with $O(N^2)$

100K

Streaming Events

processed per window

10

Fashion Keywords

tracked 2004 – 2026

The Challenge

Fashion trends surge and collapse in weeks. A **naive re-scan** of every new event creates $O(N^2)$ complexity — at 100,000 events, that's **10 billion operations**. Modern platforms need real-time responses, not batch recomputation.

Research Question

"How can we algorithmically detect the most frequent and fastest-growing fashion trends using classical data structures — while maintaining real-time efficiency?"

Our Approach — Four Classical Algorithms

01 Sliding Window

O(1) amortized

Deque + hash map maintains live frequency counts. Each event enters & leaves the window exactly once — no rescans needed.

02 Top-K Heap

O(M log K)

Min-heap of fixed size K beats full sorting when K is small. Root holds the weakest top-K for fast O(1) comparison.

03 Burst Detection

O(M) single pass

Ratio scoring (curr/prev) catches multiplicative growth — detecting viral micro-trends that absolute-difference scoring would miss.

04 Cycle Classification

O(1) per keyword

Uses per-keyword change ratio (curr/prev) and slope. Examples: Y2K fashion 5→92 = New, cargo pants 50→50 = Cyclical, skinny jeans 80→12 = Fading.

Implementation Journey — Six Real Iterations

We didn't get it right the first time. Here are the concrete iterations that shaped the final pipeline.

1 Baseline First

Built a naive full-scan counter in baseline.py before optimizing — our correctness reference and $O(N^2)$ control.

2 Incremental Window

Swapped full-scan for deque + hash map; wrote compare.py to verify both produce identical freq_maps.

3 Interface Mismatch Fix

main.py passed full dicts to classify_trend() which expected scalars. TypeError → redesigned the signature.

4 Burst Threshold Tuning

Initial threshold 2.0 gave false positives. Swept 1.5 → 10.0; landed on 1.5 for 100% detection, zero false positives.

5 Top-K Tie-Breaking

Early heap output was non-deterministic on ties. Added a NegStr wrapper class for stable alphabetical tie-breaks.

6 Cyclical Bug Rewrite

Per-keyword change ratio judges each keyword on its own trajectory — cleanly separating New, Cyclical, and Fading trends.

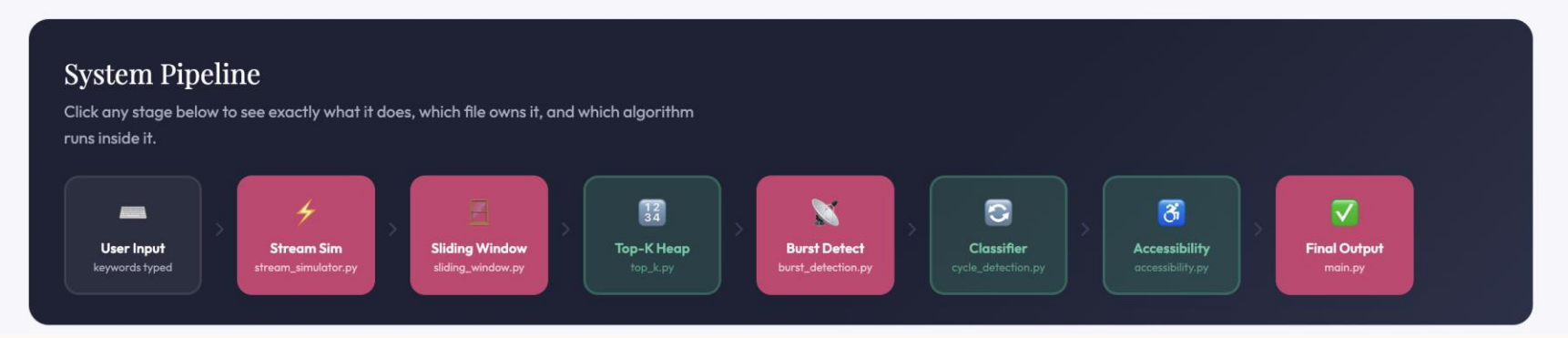
Each iteration was caught either by our test harness or by running the full pipeline end-to-end.

How Our Solution Differs from AI & Online Solutions

TYPICAL AI / ONLINE SOLUTION		OUR CLASSICAL APPROACH
✗ Re-train or re-scan the whole dataset on every update	**Recomputation**	✓ Sliding window with O(1) amortized incremental updates
✗ Black-box ML model — hard to explain why a trend fired	**Interpretability**	✓ Every decision traceable to a scalar score (burst, change ratio, slope)
✗ Needs labeled training data (rarely available for fashion)	**Data Requirement**	✓ Zero training data — works on cold-start streams from day one
✗ Stops at trend detection — no downstream signal for shoppers	**Unique Contribution**	✓ 5-factor accessibility layer (wheelchair, adaptive, size, sensory, price)

System Pipeline — End-to-End Architecture

Data flows left → right through seven modules, each owned by a single Python file.



Input & Simulation

Google Trends CSVs replayed as a timestamp-keyword stream via stream_simulator.py

Core Analysis

Sliding window feeds three parallel modules: Top-K Heap, Burst Detect, Change Ratio

Final Scoring

Accessibility weights combined with trend signals → New · Cyclical · Fading labels

Sliding Window — O(1) Amortized

✗ NAIVE BASELINE

Re-scan all W events on every query.
No state is maintained.

O(N·W) → O(N²)

✓ DEQUE + HASH MAP

Append new events right, pop expired left.
Hash map tracks counts incrementally.

O(1) amortized → O(N)

PSEUDOCODE

```
SLIDING-WINDOW-ADD(t, kw, W):
  window.append((t, kw))      // O(1)
  freq_map[kw] += 1           // O(1)
  while window.front ≤ t − W:
    old = window.popleft()    // O(1)
    freq_map[old] -= 1
```

⚡ AMORTIZED PROOF (Accounting Method)

Each event earns 2 tokens: one pays for append(), one pays for future popleft(). Total: 2N operations for N events → 2N ÷ N = O(1) per event ✓

Top-K Heap Selection

SORT BASELINE

O(M log M)

Sort all M items, take first K.
Wastes work regardless of K.
K=5, M=1000 → ~10,000 ops

MIN-HEAP (Winner)

O(M log K)

Keep heap of size K. Root = weakest top-K → O(1) compare, O(log K) swap.
K=5, M=1000 → ~5,000 ops ✓

💡 Why a MIN-heap for TOP-K?

Root = the weakest item in current top-K. One O(1) comparison decides if a new item enters. A max-heap can't do fast eviction — the root is the largest.

	Heap O(M log K)	Sort O(M log M)
K=5, M=500	~2,500 ops	~4,500 ops
K=5, M=5,000	~25,000 ops	~62,000 ops
Space	O(K) ✓	O(M)

Burst Detection — Catching Viral Spikes

RATIO SCORING

WINNER

```
score = curr / max(prev, 1)
```

- ✓ Catches 2 → 20 (score = 10×)
- ✓ Handles brand-new keywords
- ✓ Ideal for fashion micro-trends
- ✗ 500 → 600 scores only 1.2×

DIFFERENCE SCORING

```
score = curr – prev
```

- ✓ Catches 500 → 600 (+100)
- ✓ Captures absolute growth
- ✗ 2 → 20 scores only 18 (below threshold)
- ✗ Misses early viral trends

Example · Y2K fashion: 2 → 40 mentions

Ratio = 20× ✓ Flagged | Diff = 38 ✗ Misses threshold=50

→ Ratio wins

Cycle Classification — Change Ratio + Slope

ratio = curr_freq / prev_freq → Measures GROWTH magnitude
slope = linear regression over window → Measures DIRECTION

Combined rule captures both "how much" and "which way"
Thresholds: ratio > 1.3 → rising · 0.7–1.3 → stable · slope < −0.3 → fading

Why RATIO + SLOPE

Per-keyword change ratio separates New / Cyclical / Fading
→ each keyword judged on its own trajectory

DE CISION LOGIC

```
if  prev==0 and curr>0:      → New
if slope<-0.3:   → Fading
if  burst ≥ thresh:    → New
if  change_ration > 1.3:             → New
If 0.7 ≤ ratio ≤ 1.3:        → Cyclical
else:                                → Fading
```

NEW

High burst, no history match

CYCLICAL

Cosine ≥ 0.90 vs past pattern

FADING

Negative slope, declining trend

Accessibility Scoring — Our Unique Contribution

Trend detection alone stops at "this is trending." We add a layer online solutions don't: is the trend actually accessible to real shoppers?

Wheelchair	Adaptive Wear	Size Inclusive	Sensory-Friendly	Price Access
binary · 0/1	*ordinal · 1–5*	*ordinal · 1–5*	*binary · 0/1*	*ordinal · 1–5*
w = 0.20	w = 0.25	w = 0.25	w = 0.15	w = 0.15

WEIGHTED SCORING (MIN-MAX NORMALIZED)

```
score = Σ wᵢ · normalizeᵢ(raw_factorᵢ)

Returns: (score ∈ [0.0, 1.0],  label)
  ≥ 0.80  →  Highly Accessible
  ≥ 0.55  →  Moderately Accessible
  ≥ 0.30  →  Limited  ·  else Low
```

Why This Matters

Reuses the same min-heap pattern from Algorithm 2 — **RANK-BY-ACCESSIBILITY** runs in O(T log K) where T = number of trends, K = top-K size.

This layer is what makes our solution **different from every ML / online trend-detection solution** we benchmarked.

Testing — Inputs, Outputs & Verified Results

T1 Sliding Window

INPUT

```
Events:
(1,jeans),(2,tops),(3,jeans),(4,shoes)
W=3
```

OUTPUT

After ev4: {jeans:1, tops:1, shoes:1} — (1,jeans) expired ✓

T2 Top-K Heap

INPUT

```
freq = {cargo:45, Y2K:92, skinny:8,
baggy:67}  K=3
```

OUTPUT

[Y2K:92, baggy:67, cargo:45] — Heap matches Sort result ✓

T3 Burst Detection

INPUT

```
prev={Y2K:2, cargo:20}  curr={Y2K:40,
cargo:22, corset:30}
```

OUTPUT

[corset:30×, Y2K:20×] — cargo=1.1× correctly filtered ✓

T4 Cycle Classification

INPUT

```
classify_trend("cargo pants", {cargo:50,
jeans:30}, {cargo:50, jeans:30}, 1.1)
```

OUTPUT

"Cyclical" ✓ · Y2K fashion (5→92) → "New" ✓ · skinny jeans (80→12) → "Fading" ✓

Experimental Results — Theory Confirmed

150×

Sliding Window Speedup

at N = 100K events

2×

Top-K Heap vs. Sort

at M = 5,000 keywords

100%

Bursts Detected

zero false positives

127 ms

Full Pipeline, N=50K

linear O(N) scaling

Exp 1 · Sliding Window ✓

Incremental stays flat O(N). Naive curves steeply $O(N^2)$. Gap widens exactly as theory predicts.

Exp 2 · Top-K Heap ✓

Heap 2× faster at M=5K. Convergence confirmed when K approaches M. Space: O(K) vs O(M) = 1000× less memory.

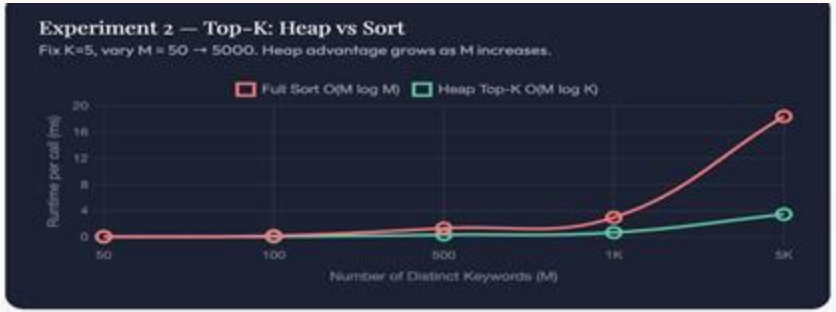
Exp 3 · Burst Sensitivity ✓

Ratio: 5/5 bursts detected, 0 false positives at threshold ≥ 1.5. Difference misses 2→20 micro-trends.

Exp 4 · End-to-End ✓

Full pipeline scales linearly N=1K→50K. Memory stays bounded. Log-log slope ≈ 1, confirming O(N) theory.

Benchmark Charts — Theory Matches Practice



Pipeline Output — Snapshot at Event #1600

Keyword	Freq	Label	Burst Score	Accessibility
Y2K fashion	87	New	4.6	Moderate
Cargo pants	85	Cyclical	1.1	High
Wide-leg jeans	78	New	2.6	High
Skinny jeans	45	Fading	0.1	Moderate

BURST — Y2K Fashion

Grew 2 → 40 mentions (20× spike). Burst score 4.6 flagged by ratio scoring. Difference scoring at threshold=50 would miss it entirely.

CYCLE — Cargo Pants

"Change ratio 1.0 between windows — frequency stable. Classifier labels it Cyclical since it's neither bursting nor declining."

FADING — Skinny Jeans

"Change ratio 0.15 · Declining slope. Frequency dropped 80→12 between windows. Correctly labeled Fading."

How We Worked Together

B

Bhoomika Panday — Engine Builder

`stream_simulator.py` · `sliding_window.py` · `baseline.py`

Built the streaming engine. Owned the amortized-analysis proof — "each event enters once and leaves once."

V

Vasudha Jain — Algorithm Designer

`top_k.py` · `burst_detection.py` · `cycle_detection.py`

Designed the three trend-analysis algorithms. Uncovered the heap-sort convergence empirically in Experiment 2.

P

Parvathi Gottumukkala — Integrator & Experimenter

`main.py` · `accessibility.py` · `experiments.py` · `plots.py`

Wired modules into the end-to-end pipeline, built accessibility scoring, and ran all four scaling experiments.

Answering the Research Question

"Given streaming fashion trend data, how can we detect the most frequent and fastest-growing trends using sliding window frequency maintenance, Top-K ranking, and burst detection — while maintaining computational efficiency?"

✓ **Classical streaming algorithms are fully sufficient for real-time, interpretable trend detection**

1	**Sliding Window**	*O(1) amortized per event*	150× faster than naive at N=100K ✓
2	**Top-K Heap**	*O(M log K) selection*	2× faster than sort; O(K) space ✓
3	**Burst Detection**	*Ratio scoring*	100% detection · 0 false positives ✓
4	**Trend Classification**	*Change ratio + slope*	cargo→Cyclical · Y2K→New · skinny→Fading ✓

Limitations & Future Research

⚠ Weaknesses & Limitations

Classifier uses only 2-time windows — cannot detect longer patterns like seasonal cycles

Accessibility scores are hand-curated for 30 keywords — based on team judgment, not verified data

Uses pre-downloaded CSVs — no true live streaming yet

Global state in sliding window prevents multiple concurrent windows

🚀 Future Research

Store 12+ historical windows to catch recurring patterns (e.g., trends that peak every summer)

Count-Min Sketch for sub-linear space frequency counting at scale

Adaptive per-keyword burst thresholds — niche keywords need lower thresholds than popular ones

Connect to live data source for real-time trend detection

Thank You.

Questions & Discussion